1. INTRODUCTION

1.1 Project Introduction

With the increasing use of online recruitment platforms, organizations are receiving a large number of job applications for every vacancy. In many cases, a single job role attracts hundreds of resumes, making the screening process difficult for recruiters. Manually reviewing each resume and comparing it with job requirements takes a lot of time and effort. Therefore, there is a strong need for an automated system that can simplify and speed up this process.

The AI Resume Matcher is developed as a smart solution to automate resume screening and candidate evaluation. This system makes use of Natural Language Processing (NLP) along with machine learning-based techniques to understand and process the content of resumes. It extracts relevant details such as skills, qualifications, and experience, and compares them with the give job description to determine how well a candidate fits the role.

The primary goal of this project is to support recruiters by reducing manual work and improving the efficiency of the hiring process. In this system, resumes are automatically analyzed, and candidates are ranked based on their relevance to the job description. This helps recruiters focus only on the most suitable applicants instead of going through every resume individually.

In our implementation, resumes are uploaded into the system and processed using a parsing mechanism that extracts text from PDF documents. The extracted content is then transformed into vector representations using a Sentence Transformer model, which helps in capturing the contextual meaning of the text. After generating embeddings for both resumes and job descriptions, cosine similarity is applied to calculate how closely they match. Based on this similarity score, candidates are ranked accordingly.

2. WORKING ENVIRONMENT

2.1 Hardware Requirement

- Processor: Intel i3 / i5 or above
- RAM: Minimum 8 GB
- Storage: 256 GB or above
- System: Laptop or Desktop

2.2 Software Requirement

- Operating System: Windows / Linux / macOS
- Programming Language: Python
- Framework: FastAPI / Flask
- Frontend Technologies: HTML, CSS, JavaScript
- Database: SQLite / PostgreSQL
- Libraries Used: Sentence Transformers, PyPDF2, NumPy, Scikit-learn

2.3 System Software

System software forms the foundation for developing and running any application efficiently. It provides the necessary environment for executing programs and managing system resources. In this project, system software includes the operating system, development tools, and supporting platforms required to build the AI Resume Matcher.

The application is developed and executed primarily on the Windows operating system, which offers a user-friendly and stable environment. For coding and development, Visual Studio Code is used as the main editor, as it provides useful features such as code suggestions, debugging support, and extensions that simplify Python development.

Python is chosen as the core programming language due to its simplicity and strong support for Artificial Intelligence and Natural Language Processing tasks. Various libraries such as Sentence Transformers, NumPy, and Scikit-learn are integrated into the system to perform text representation and similarity analysis effectively. These libraries help in converting textual data into meaningful numerical formats and comparing them accurately.

Version control is managed using Git, which allows tracking of code changes and helps in maintaining different versions of the project. Overall, the combination of these system software components ensures a smooth development process and reliable execution of the AI Resume Matcher system.

In addition to the core development tools, the project setup also includes virtual environment management to handle dependencies efficiently. By using virtual environments, all required libraries and packages are maintained separately for this project, avoiding conflicts with other applications on the system. This ensures better stability and reproducibility of the project. Furthermore, package management tools such as pip are used to install and update the necessary libraries. These practices make the development process more organized and help in deploying the application smoothly across different systems without compatibility issues.

3. SYSTEM ANALYSIS

3.1 Feasibility Study

A feasibility study is an important step in system analysis that determines whether the proposed system is practical and beneficial for implementation. It evaluates different aspects such as technical feasibility, economic feasibility, and operational feasibility. By conducting a feasibility study, developers can understand whether the system can be developed using the available resources and technologies. It also helps organizations determine whether the project will provide value and improve existing processes. In this project, the feasibility study ensures that the AI Resume Matcher system can be successfully developed and deployed. The study confirms that the system is capable of improving the recruitment process.

Table 3.1.1 Feasibility Study:

Type	**Tools Used**	**Description**	**Result**
Technical	Python, NLP	AI system	Feasible
Economic	Open-source	Low cost	Low Cost
Operational	UI system	Easy use	Efficient
Resource	Basic system	No hardware	Available
Time	Automation	Fast process	Fast
Risk	Stable tech	Low risk	Safe

3.2 Existing System

In the traditional recruitment process, organizations usually rely on manual resume screening to identify suitable candidates. Recruiters receive a large number of resumes for a single job position and must review each resume individually. This process requires significant time and effort because recruiters must carefully read through each document and compare it with the job requirements. When the number of applicants is very high, this process becomes even more difficult. As a result, the recruitment process may take several days or even weeks to complete. This highlights the limitations of the existing recruitment system.

The existing system mainly depends on human judgment and manual analysis of resumes. Recruiters often search for specific keywords such as skills, experience, and educational qualifications while reviewing resumes. However, this approach may not always identify the most suitable candidates because some candidates may describe their skills using different terms. As a result, potentially qualified candidates may be overlooked during the screening process. This limitation reduces the effectiveness of manual resume evaluation.

Table 3.2 Existing vs ProposedTable :

Feature	**Existing**	**Proposed**	**Benefit**
Screening	Manual	AI-based	Faster
Time	High	Low	Saves time
Accuracy	Low	High	Better
Bias	Yes	Reduced	Fair
Scalability	Limited	High	Handles more
Matching	Keywords	NLP	Smart match

3.3 Drawbacks of Existing System

The existing manual resume screening system has several drawbacks that affect the efficiency and accuracy of the recruitment process. One of the major disadvantages is the amount of time required to review each resume. Recruiters must read through many resumes before identifying suitable candidates. This process becomes extremely time-consuming when the number of applicants is very large. As a result, the recruitment process may become slow and inefficient.

Another drawback of the existing system is the possibility of human bias during candidate selection. Recruiters may unintentionally favor certain candidates based on personal preferences or subjective judgment. This can lead to unfair hiring decisions and may prevent qualified candidates from being considered. Bias in recruitment can also negatively impact diversity within organizations.

Manual resume screening is also prone to errors and inconsistencies. Recruiters may overlook important information while reviewing resumes due to fatigue or time pressure. Sometimes resumes that contain relevant skills may be rejected simply because the keywords used in the resume differ from those expected by the recruiter. These issues reduce the accuracy of candidate evaluation.

Another limitation of the existing system is the lack of scalability. When organizations receive a large number of applications, the manual screening process becomes difficult to manage. Recruiters may not have enough time to properly evaluate every resume. As a result, some potential candidates may never be reviewed.

Due to these drawbacks, the traditional recruitment process is often inefficient and unreliable. Organizations require a more advanced solution that can analyze resumes automatically and provide accurate candidate recommendations. This need leads to the development of AI-based resume matching systems.

3.4 Proposed System

The proposed system introduces an AI-based solution that automates the process of resume screening and candidate matching. The AI Resume Matcher system is designed to analyze resumes and compare them with job descriptions using Natural Language Processing techniques. By automating this process, the system significantly reduces the time required to evaluate candidate applications. This allows recruiters to focus on making better hiring decisions rather than manually reviewing resumes.

In the proposed system, candidates upload their resumes through a user interface. The system then processes the uploaded resume using a resume parsing module. This module extracts the textual content from the resume and prepares it for further analysis. The extracted text is cleaned and organized to ensure accurate processing by the AI model.

After extracting the resume text, the system converts both the resume content and the job description into numerical representations called embeddings. These embeddings are generated using a sentence transformer model. Embeddings help the system understand the semantic meaning of the text instead of relying only on keyword matching.

The system then calculates the similarity between the resume and job description using cosine similarity. This similarity score represents how closely the candidate's profile matches the job requirements. Candidates with higher similarity scores are ranked higher in the results list.

The proposed system provides a faster and more efficient way to identify suitable candidates. By automating resume analysis, the system reduces manual effort and improves the accuracy of candidate selection. This makes the recruitment process more efficient and reliable.

3.5 Benefits of Proposed System

The AI Resume Matcher system offers several advantages compared to traditional recruitment methods. One of the most significant benefits is the automation of resume screening. Instead of manually reviewing each resume, recruiters can rely on the system to analyze resumes and generate similarity scores. This greatly reduces the time required to identify suitable candidates.

Another important benefit is improved accuracy in candidate matching. The system uses Natural Language Processing techniques to understand the semantic meaning of resumes and job descriptions. This allows the system to identify candidates whose skills and experience closely match the job requirements. As a result, recruiters can make better hiring decisions.

The system also helps reduce human bias in the recruitment process. Since candidate evaluation is based on algorithmic analysis, all resumes are assessed using the same criteria. This ensures a fair and objective evaluation process for all applicants. It also improves transparency in candidate selection.

In addition, the system increases productivity for HR professionals. Recruiters no longer need to spend hours reviewing resumes manually. Instead, they can quickly view ranked candidate results and focus on interviewing the most suitable applicants. This improves overall recruitment efficiency.

Overall, the proposed system enhances the recruitment process by saving time, improving accuracy, and reducing manual workload. Organizations can use the system to handle large numbers of applications more effectively and identify the best candidates more quickly.

3.6 Scope of the Project

The scope of the AI Resume Matcher project focuses on developing an intelligent system that automates resume screening using Artificial Intelligence and Natural Language Processing techniques. The system is designed to analyze resumes and compare them with job descriptions to determine candidate suitability. By using AI-based analysis, the system helps recruiters quickly identify candidates whose skills match the job requirements.

The project primarily focuses on extracting textual information from resumes and analyzing the content using machine learning techniques. The system converts both resumes and job descriptions into embeddings using a sentence transformer model. These embeddings represent the semantic meaning of the text and allow the system to compare candidate profiles effectively.

Another important aspect of the project scope is the ranking of candidates based on similarity scores. After calculating the similarity between resumes and job descriptions, the system ranks candidates according to their relevance. This allows recruiters to easily identify the most suitable candidates for a particular job position.

The system is designed to support resumes in PDF format and process them automatically through the resume parsing module. The extracted information is then used by the AI matching module to generate similarity scores. This automation simplifies the resume evaluation process.

In the future, the scope of the project can be expanded by integrating additional features such as job portal integration, multi-language resume support, and advanced AI models for improved candidate analysis. These enhancements can further improve the efficiency and capabilities of the system.

4. SYSTEM DESIGN

4.1 Data Flow Diagram

A Data Flow Diagram (DFD) is used to represent how data moves within the system and how it is processed at different stages. It helps in understanding the overall workflow of the application and shows how information flows between different components. The diagram provides a clear view of the input, processing, and output stages of the system. In the AI Resume Matcher system, the DFD explains how resumes are uploaded, processed, and matched with job descriptions. This diagram helps developers and users understand how the system operates internally. It also simplifies the explanation of complex system processes.

The first step in the data flow process begins when a recruiter uploads resumes into the system. The uploaded resumes are stored in the system and sent to the resume parsing module. This module extracts the textual information from the resume documents, usually in PDF format. The extracted text is then cleaned and preprocessed to remove unnecessary characters and formatting.

Table 4.1 Process Table :

Component	Input	Process	Output
UI	Resume, JD	User action	Data sent
Backend	Request	Processing	Data handled
AI Module	Text	Matching	Score
Database	Data	Store	Records
Parser	PDF/Text	Extract	Clean text
Ranking	Score	Sort	Ranked list

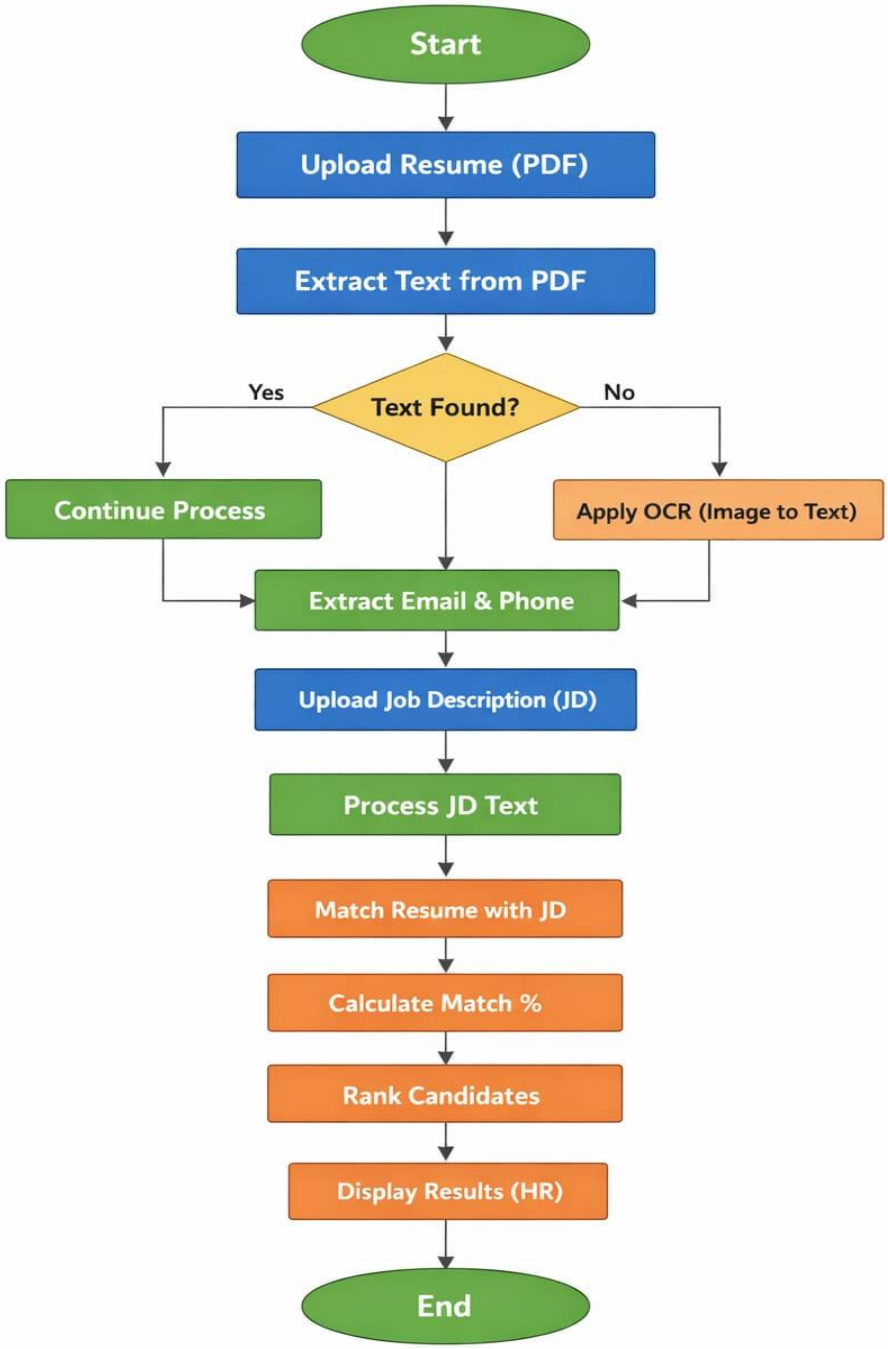

Fig 4.1.1 Flow Chart Diagram

4.2 Architecture Diagram

The system architecture describes the structure of the AI Resume Matcher application and the interaction between its components. It provides a high-level overview of how different modules work together to perform resume matching. The architecture helps developers understand the design of the system and ensures that all components communicate effectively. In this project, the architecture consists of several layers including the user interface, backend processing system, AI model layer, and database layer. Each component plays an important role in the functioning of the system. Together, these components form a complete resume matching platform.

The first layer of the architecture is the user interface layer. This layer allows users to interact with the system through a web-based interface. Recruiters can upload resumes, enter job descriptions, and view the candidate matching results through this interface. The user interface is designed to be simple and user-friendly so that recruiters can easily use the system.

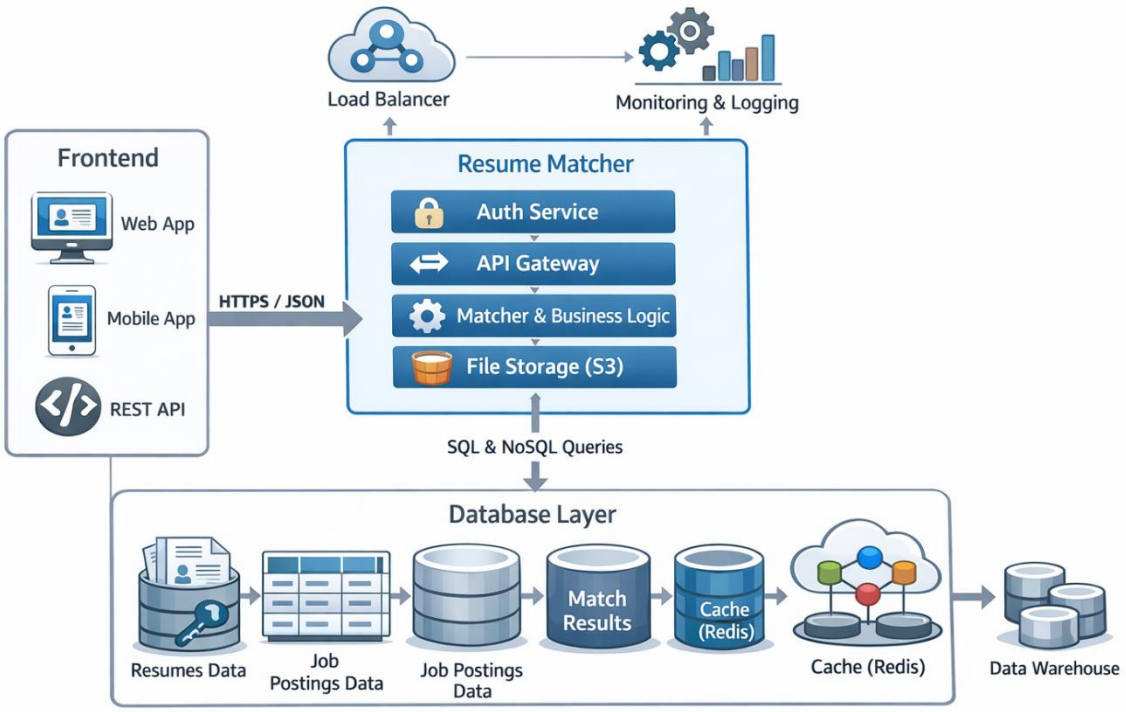

Fig 4.2.1 Architecture Diagram

4.3 Database Design

Database design is an essential component of any software system because it determines how data is stored, organized, and managed. In the AI Resume Matcher system, the database is used to store candidate information, resumes, job descriptions, and matching results. A well-designed database ensures efficient data storage and retrieval. It also helps maintain data consistency and integrity within the system. Proper database design improves system performance and reliability. Therefore, it plays a crucial role in the overall functionality of the application.

The database contains tables that store candidate details such as name, email address, skills, and resume file information. When a candidate uploads a resume, the system extracts the textual content and stores relevant details in the database. This allows the system to keep track of all candidate profiles.

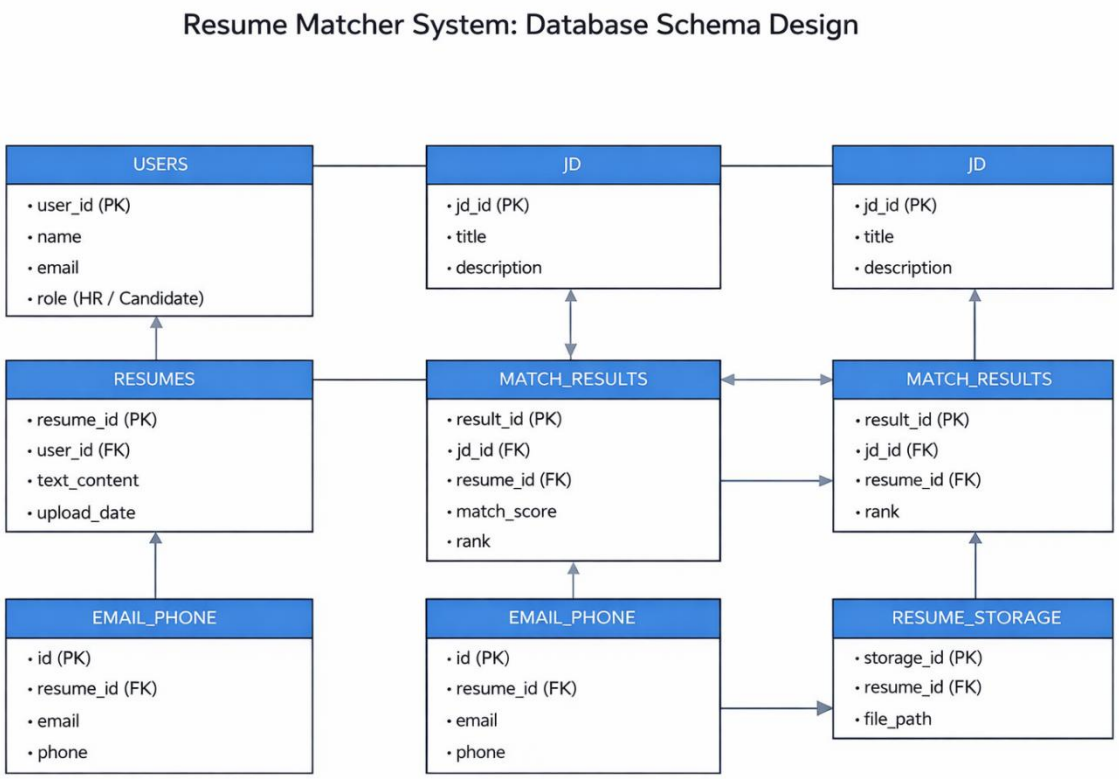

Fig 4.3.1 Database Design

5. PROJECT DESCRIPTION

5.1 Objective

The main objective of the AI Resume Matcher project is to develop an intelligent system that automates the process of resume screening and candidate selection. In traditional recruitment systems, recruiters manually review numerous resumes to identify suitable candidates. This process requires a large amount of time and effort, especially when there are hundreds of applications for a single job position. The proposed system aims to simplify this process by automatically analyzing resumes and comparing them with job descriptions. By using Artificial Intelligence techniques, the system can quickly determine the relevance of each resume. This helps recruiters make faster and more informed hiring decisions.

Another objective of the project is to improve the accuracy of candidate matching. Manual resume screening often relies on keyword searching and human judgment, which may not always identify the most suitable candidates

Table 5.1.1 Module Description :

Module	**Input**	**Process**	**Output**
Upload	Resume	Store file	Saved data
Parsing	PDF/Text	Extract text	Clean text
Matching	Resume+JD	Compare	Score
Ranking	Scores	Sort	List
Email	Details	Send mail	Notification
Display	Results	Show data	UI output

5.2 Module Description

The AI Resume Matcher system consists of several modules that work together to perform resume analysis and candidate matching. Each module is responsible for a specific task within the system. These modules ensure that the system functions efficiently and processes resumes in a structured manner. By dividing the system into multiple modules, the overall design becomes easier to understand and maintain. Each module interacts with other modules to complete the resume matching process. This modular approach improves the flexibility and scalability of the system.

The first module of the system is the Resume Upload Module. This module allows users or recruiters to upload resumes into the system through a web interface. The uploaded resumes are usually in PDF format and are stored in the system for further processing. The module ensures that the uploaded files are properly stored and accessible by other system components. It also verifies that the uploaded documents are valid and ready for analysis.

The second module is the Resume Parsing Module. This module extracts textual information from the uploaded resumes. Since resumes are usually stored in document formats such as PDF, the system must convert them into plain text for analysis. The parsing module performs this task by extracting relevant information such as skills, education, and experience. This extracted text is then prepared for further processing by the AI model.

Another important module is the AI Matching Module. This module converts the textual data from resumes and job descriptions into embeddings using a sentence transformer model. These embeddings represent the semantic meaning of the text. The module then calculates the similarity between the resume and job description using cosine similarity. This process allows the system to determine how closely a candidate's profile matches the job requirements.

The final module is the Ranking and Result Module. This module displays the similarity scores generated by the AI matching module. Based on these scores, candidates are ranked according to their relevance to the job description. Recruiters can view the ranked candidate list and identify the most suitable applicants. This module provides the final output of the system.

5.3 Implementation

The implementation of the AI Resume Matcher system is carried out using Python programming language and various Natural Language Processing libraries. Python is widely used in AI development because of its simplicity and powerful library ecosystem. The system uses libraries such as Sentence Transformers, NumPy, and Scikit-learn to perform text processing and similarity calculations. These libraries enable the system to analyze large amounts of textual data efficiently. The implementation process involves several stages including resume parsing, embedding generation, and similarity computation.

The first step in implementation involves creating the user interface that allows recruiters to upload resumes and enter job descriptions. The frontend interface is designed using web technologies such as HTML, CSS, and JavaScript. This interface communicates with the backend server to send user inputs for processing. The backend server receives the uploaded resumes and job descriptions and processes them accordingly.

The next step involves extracting textual information from the uploaded resumes. Since resumes are typically stored in PDF format, the system uses a resume parsing module to extract text from the document. The extracted text is cleaned and preprocessed to remove unnecessary characters and formatting. This preprocessing step ensures that the text is suitable for analysis by the AI model.

After preprocessing the text, the system generates embeddings using a sentence transformer model. These embeddings convert textual data into numerical vectors that represent the semantic meaning of the text. The embeddings for the resume and job description are then compared using cosine similarity. This calculation determines how closely the candidate's resume matches the job requirements.

Finally, the similarity scores are used to rank candidates based on their relevance to the job description. The results are displayed through the system interface so that recruiters can review them easily. This implementation process allows the AI Resume Matcher system to perform automated resume screening efficiently.

6. SYSTEM TESTING

6.1 Testing Definition

Testing is an essential phase in the software development lifecycle that ensures the system functions correctly and meets user requirements. It involves evaluating the software to identify errors, bugs, and performance issues. By performing testing, developers can verify whether the system behaves as expected under different conditions. Testing helps improve the reliability and stability of the application before it is deployed. In the AI Resume Matcher system, testing ensures that all modules operate correctly and produce accurate results.

The main purpose of testing is to verify that the system processes resumes correctly and generates accurate similarity scores. Each module of the system must be tested individually to ensure that it performs its intended function. For example, the resume parsing module must correctly extract text from PDF documents. Similarly, the AI matching module must accurately compute similarity scores between resumes and job descriptions.

Testing also ensures that the system handles different types of inputs without errors. Users may upload resumes in different formats or provide job descriptions with varying structures. The system must be able to process these inputs without affecting its performance. Proper testing ensures that the application can handle such scenarios effectively.

Another important aspect of testing is verifying the user interface functionality. The system interface should allow users to upload resumes, input job descriptions, and view results without difficulty. Testing ensures that all interface components work smoothly and provide a good user experience.

Overall, testing improves the quality of the AI Resume Matcher system by identifying and fixing potential issues. This ensures that the system performs reliably and provides accurate results during real-world usage.

6.2 Testing Objective

The primary objective of testing is to ensure that the system meets all functional and performance requirements. Testing helps verify that the AI Resume Matcher system operates correctly and produces reliable outputs. By conducting thorough testing, developers can ensure that the system performs its intended tasks without errors. This process helps maintain the overall quality of the software. Proper testing also ensures that the system behaves consistently in different usage scenarios. As a result, users can rely on the system for accurate resume analysis and candidate matching.

One of the key objectives is to validate the resume parsing functionality. The system must correctly extract textual information from uploaded resumes. If the text extraction process fails or produces incorrect results, the similarity analysis may also become inaccurate. Therefore, testing ensures that resume parsing works effectively. The system must be tested with different types of resume formats to confirm that the text extraction process remains consistent. Proper validation of this module ensures that all resume data is correctly prepared for further analysis.

Another objective of testing is to evaluate the similarity calculation process. The system must correctly compute similarity scores between resumes and job descriptions using cosine similarity. These scores determine the ranking of candidates in the system. Accurate similarity calculations are essential for reliable candidate recommendations. Testing ensures that the AI matching module produces consistent and meaningful similarity results. This helps maintain the accuracy and reliability of the candidate ranking system.

Testing also aims to ensure that the system performs efficiently even when processing multiple resumes. The system should be able to analyze several resumes without slowing down or producing errors. Performance testing helps evaluate how well the system handles large volumes of data. The system must be capable of processing multiple resumes within a reasonable time frame. This ensures that recruiters can obtain results quickly without experiencing delays during the recruitment process.

6.3 Types of Testing

Different types of testing are used to ensure that the AI Resume Matcher system functions properly. Each type of testing focuses on evaluating specific aspects of the system. By performing multiple testing methods, developers can ensure that the system operates reliably and efficiently.

Unit testing is the first stage of testing where individual components of the system are tested separately. For example, the resume parsing module and similarity calculation module are tested independently to verify that they perform their functions correctly. Unit testing helps identify errors within individual modules before they are integrated into the system.

Integration testing is performed after unit testing to ensure that different modules work together correctly. In the AI Resume Matcher system, integration testing verifies the interaction between modules such as resume upload, parsing, and similarity calculation. This testing ensures that data flows smoothly between different components of the system.

System testing evaluates the complete system to ensure that all functionalities operate as expected. This includes testing the entire workflow from resume upload to candidate ranking. System testing verifies that the application behaves correctly under normal usage conditions.

User acceptance testing is the final stage of testing where real users evaluate the system to ensure it meets their requirements. Recruiters can test the system by uploading resumes and reviewing candidate rankings. Their feedback helps determine whether the system is ready for deployment.

7. CONCLUSION

7.1 Summary

The AI Resume Matcher system demonstrates how Artificial Intelligence can be used to improve the recruitment process. In traditional recruitment systems, recruiters must manually review a large number of resumes to identify suitable candidates. This process is time-consuming and requires significant effort. The proposed system automates this process by analyzing resumes and matching them with job descriptions using Natural Language Processing techniques. By using AI-based analysis, the system can quickly identify candidates whose skills match the job requirements. This improves both the speed and efficiency of the recruitment process.

The system uses advanced techniques such as sentence transformers and cosine similarity to analyze the relationship between resumes and job descriptions. These techniques allow the system to understand the semantic meaning of textual information rather than relying only on simple keyword matching. As a result, the system can identify relevant candidates more accurately. The use of embeddings helps represent textual data in numerical form, making it easier for the system to compare candidate profiles with job requirements.

Another important aspect of the system is the automation of resume processing. The system extracts textual information from resumes using a resume parsing module. After extracting the text, it processes the information and converts it into embeddings for similarity analysis. This automated process eliminates the need for manual resume evaluation and significantly reduces recruiter workload. Recruiters can simply upload resumes and view the ranked candidate results.

The AI Resume Matcher system also improves fairness in the recruitment process. Since candidate evaluation is based on algorithmic analysis rather than personal judgment, all applicants are assessed using the same criteria. This helps reduce human bias in candidate selection. It also ensures that candidates are evaluated based on their skills and qualifications rather than subjective factors.

7.2 Future Enhancement

Although the AI Resume Matcher system provides an effective solution for automated resume screening, there are several opportunities for further improvement. One possible enhancement is the integration of the system with online job portals. By connecting the system with recruitment platforms, organizations can automatically collect resumes from job applicants. This would allow the system to analyze large volumes of applications more efficiently. Such integration would also make the recruitment process more streamlined.

Another potential improvement is the use of more advanced AI models for candidate analysis. Large language models and deep learning techniques can provide deeper understanding of resume content and job descriptions. These models can identify complex relationships between candidate skills and job requirements. By using advanced AI techniques, the system can further improve the accuracy of candidate recommendations.

The system can also be enhanced by supporting multiple resume formats and languages. Currently, many recruitment systems primarily process resumes written in a specific format or language. By adding support for multiple languages and document formats, the system can be used by organizations operating in different regions. This enhancement would make the system more flexible and globally applicable.

Another useful feature that can be added in the future is automated candidate communication. The system could automatically send email notifications to candidates regarding application status or interview invitations. This would further automate the recruitment workflow and reduce manual communication tasks for recruiters.

In the future, the system can also include advanced analytics features that provide insights into candidate trends and hiring patterns. Recruiters could use these insights to make more informed hiring decisions. These enhancements would make the AI Resume Matcher system more powerful and valuable for organizations.

8. APPENDIX

8.1 Screenshots

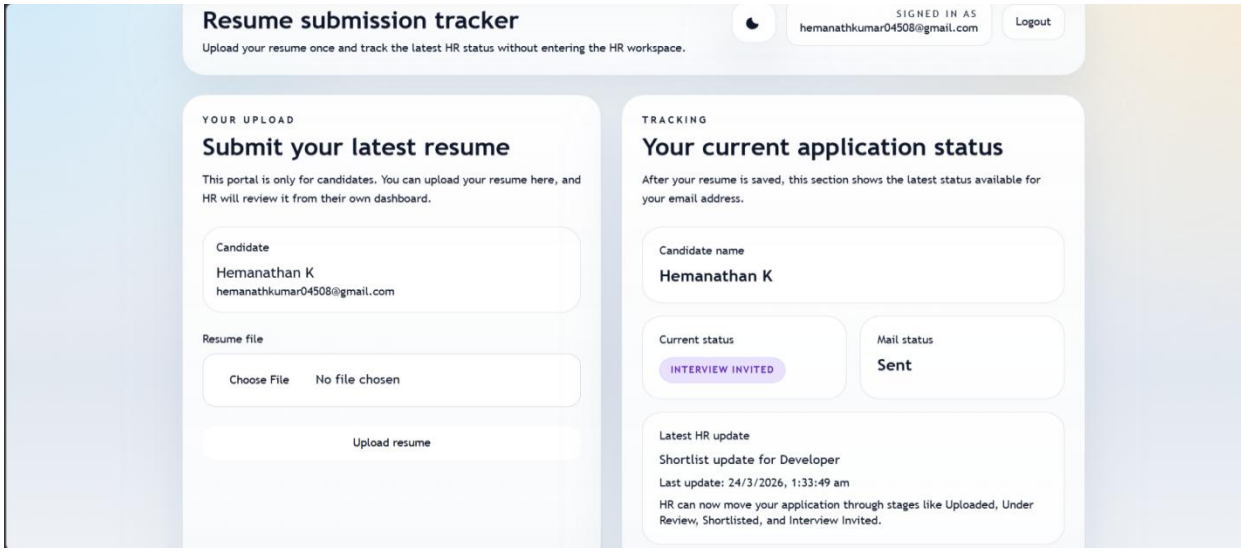

Fig 8.1.1 Candidate Portal

(The Candidate Portal allows users to upload their resumes and provide necessary details such as email and phone number.)

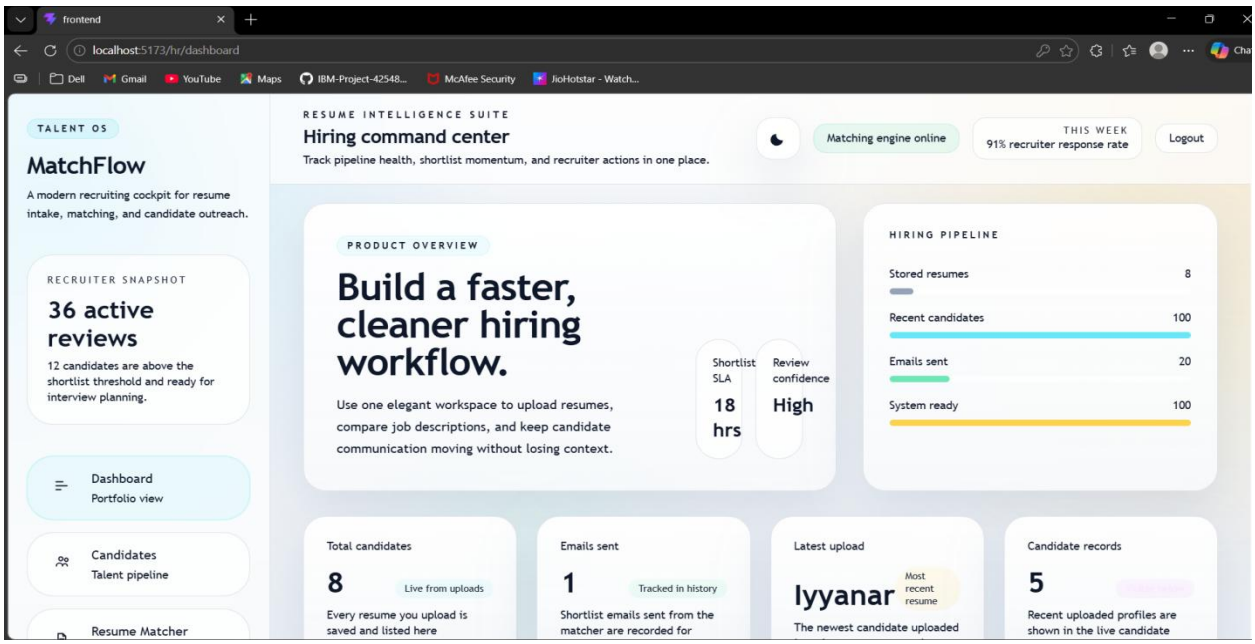

Fig 8.1.2HR Dashboard

(The HR Dashboard provides an interface for recruiters to upload job descriptions and manage candidate data.)

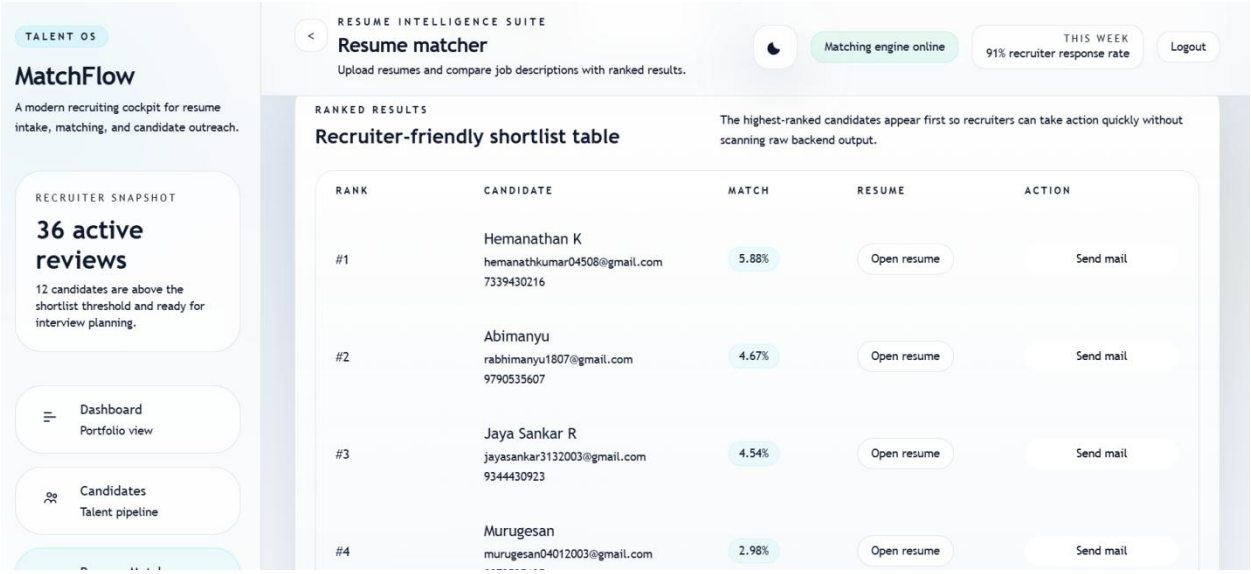

Fig 8.1.3Candidates Matching

(The Candidates Matching module compares resumes with the job description using a matching algorithm.)

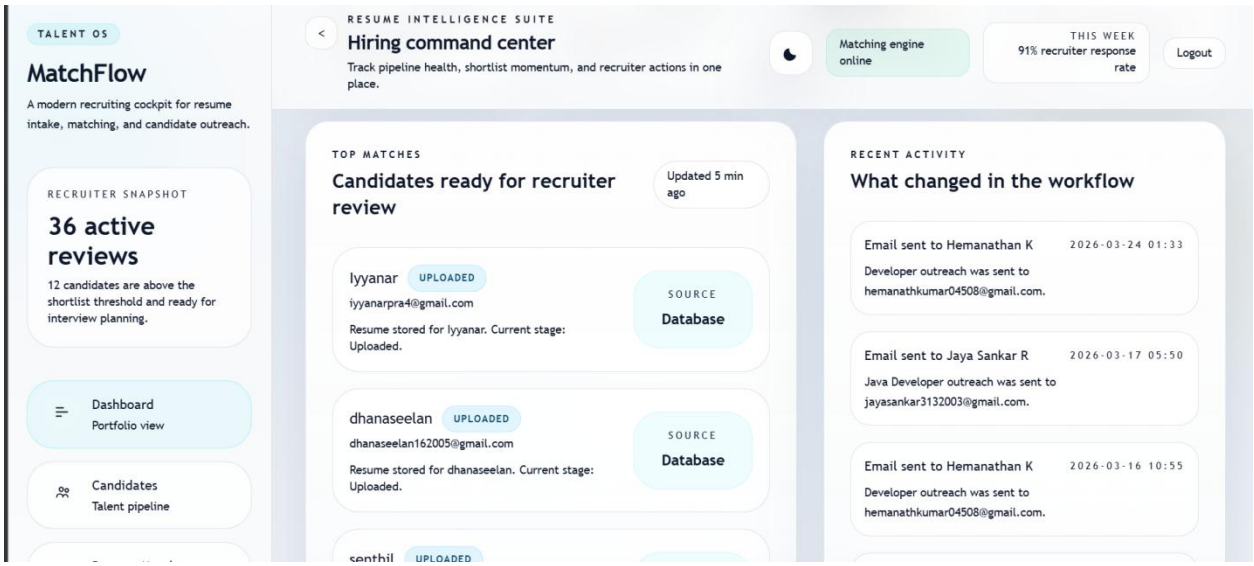

Fig 8.1.4Sending mail

(The Sending Mail feature allows HR to contact shortlisted candidates directly from the system.)

8.2 Coding :

```
from fastapi import FastAPI, UploadFile, File, Form

from fastapi.staticfiles import StaticFiles

from sqlalchemy.orm import Session

from db import SessionLocal, engine

import models

from matcher import match_percentage

from parser import extract_contact_info, extract_job_role

from auto_mail import send_shortlist_mail

from pydantic import BaseModel


import fitz, pytesseract, io, os, uuid

from PIL import Image

import shutil

from dotenv import load_dotenv


load_dotenv()

PUBLIC_URL = os.getenv("PUBLIC_URL", "http://127.0.0.1:8005")


# Tesseract path for OCR

import pytesseract

pytesseract.pytesseract.tesseract_cmd = r"C:\Program Files\Tesseract-OCR\tesseract.exe"
```

```
models.Base.metadata.create_all(bind=engine)

app = FastAPI()

UPLOAD_FOLDER = "uploads"
os.makedirs(UPLOAD_FOLDER, exist_ok=True)
app.mount("/files", StaticFiles(directory="uploads"), name="files")

# ================= PDF TEXT =================
def extract_text_from_pdf(contents):
    text = ""
    doc = fitz.open(stream=contents, filetype="pdf")

    for page in doc:
        page_text = page.get_text()

        if page_text.strip():
            text += page_text
        else:
            pix = page.get_pixmap(matrix=fitz.Matrix(2,2))
            img = Image.open(io.BytesIO(pix.tobytes())).convert("L")
            text += pytesseract.image_to_string(img)

    return text
```

```
# ================= UPLOAD RESUME =================
@app.post("/upload_resume/")
async def upload_resume(name: str = Form(...), file: UploadFile = File(...)):

    contents = await file.read()
    unique_name = f"{uuid.uuid4()}_{file.filename}"

    with open(f"{UPLOAD_FOLDER}/{unique_name}", "wb") as f:
        f.write(contents)

    text = extract_text_from_pdf(contents) if file.filename.endswith(".pdf") else contents.decode()

    email, phone = extract_contact_info(text)

    db = SessionLocal()

    if not email:
        return {"msg":"Email not found"}

    existing = db.query(models.Resume).filter(
        (models.Resume.email == email) |
        (models.Resume.phone == phone)
    ).first()

    if existing:
```

```
        return {"msg":"Resume exists"}

    db.add(models.Resume(
        name=name,
        email=email,
        phone=phone,
        resume_text=text,
        file_path=unique_name
    ))

    db.commit()
    db.close()

    return {"msg":"Resume uploaded"}

# ================= MATCH JD =================
@app.post("/match_jd/")
async def match_jd(file: UploadFile = File(...)):

    contents = await file.read()
    jd_text = extract_text_from_pdf(contents) if file.filename.endswith(".pdf") else
contents.decode()

    job_role = extract_job_role(jd_text)
    jd_text_lower = jd_text.lower()
```

```
db = SessionLocal()
resumes = db.query(models.Resume).all()

results = []

for r in resumes:
    score = match_percentage(jd_text_lower, (r.resume_text or "").lower())

    results.append({
        "name": r.name,
        "email": r.email,
        "match": score,
        "resume_link": f"{PUBLIC_URL}/files/{r.file_path}",
        "phone": r.phone
    })

db.close()

results.sort(key=lambda x: x["match"], reverse=True)

for i,r in enumerate(results):
    r["rank"] = i+1

return {
```

```
        "detected_role": job_role,
        "results": results
    }

# ================= SEND MAIL =================
class MailRequest(BaseModel):
    email: str
    name: str
    job_role: str

@app.post("/send_mail/")
def send_mail(data: MailRequest):

    print("MAIL SENT TO:", data.email)

    send_shortlist_mail(
        to_email=data.email,
        candidate_name=data.name,
        job_role=data.job_role
    )

    return {"msg":"Mail sent successfully"}
```

App.py :

```
import os
from dotenv import load_dotenv
import streamlit as st
import requests
import pandas as pd

# ---------------- ENV ----------------
load_dotenv()

API_URL = os.getenv("API_URL", "http://127.0.0.1:8000")

st.set_page_config(page_title="AI Resume Matcher", layout="wide")

# ---------------- SESSION ----------------
if "role" not in st.session_state:
    st.session_state.role = None

if "results" not in st.session_state:
    st.session_state.results = None

if "job_role" not in st.session_state:
    st.session_state.job_role = None

# ---------------- LOGIN SCREEN ----------------
```

```
if st.session_state.role is None:

    st.title("🔐 Login")

    col1, col2 = st.columns(2)

    with col1:
        if st.button("👤 Login as Candidate", use_container_width=True):
            st.session_state.role = "candidate"
            st.rerun()

    with col2:
        if st.button("👨‍💼 Login as HR", use_container_width=True):
            st.session_state.role = "hr"
            st.rerun()

# ---------------- CANDIDATE UI ----------------
elifst.session_state.role == "candidate":

    st.title("👤 Candidate Portal")

    if st.button("⬅ Logout"):
        st.session_state.role = None
        st.rerun()
```

```
st.header("Upload Your Resume")

name = st.text_input("Full Name*")
resume_file = st.file_uploader("Upload Resume*", type=["pdf", "txt"])

if st.button("Submit Resume"):

    if name and resume_file:

        res = requests.post(
            f"{API_URL}/upload_resume/",
            data={"name": name},
            files={"file": (resume_file.name, resume_file, resume_file.type)}
        )

        if res.status_code != 200:
            st.error(res.text)
            st.stop()

        try:
            data = res.json()
            st.success(data.get("msg", "Resume Uploaded Successfully"))
        except:
            st.error("Invalid response from server")
```

```
        else:
            st.warning("Enter name and upload resume")

# ---------------- HR UI ----------------
elifst.session_state.role == "hr":

    st.title("👩‍💼 HR Dashboard")

    if st.button("⬅ Logout"):
        st.session_state.role = None
        st.session_state.results = None
        st.rerun()

    st.header("Upload Job Description")

    jd_file = st.file_uploader("Upload JD", type=["pdf", "txt"])

    # ---------------- MATCH BUTTON ----------------
    if st.button("Match Candidates"):

        if not jd_file:
            st.warning("⚠ Upload JD first")
            st.stop()

        res = requests.post(
```

```
        f"{API_URL}/match_jd/",
        files={"file": (jd_file.name, jd_file, jd_file.type)}
    )

    if res.status_code != 200:
        st.error(res.text)
        st.stop()

    try:
        response = res.json()
    except:
        st.error("Invalid API response")
        st.stop()

    if "results" not in response:
        st.error("❌ API error")
        st.stop()

    st.session_state.results = response["results"]
    st.session_state.job_role = response.get("detected_role", "Job")

# ---------------- SHOW RESULTS ----------------
if st.session_state.results is not None:

    data = st.session_state.results
```

```
df = pd.DataFrame(data)

df = df.rename(columns={
    "name": "Candidate 👤",
    "match": "Match % 🎯",
    "resume_link": "Resume 📄",
    "phone": "Phone 📱"
})

df = df.sort_values("Match % 🎯", ascending=False)

st.subheader("🏆 Top Matching Candidates")

st.dataframe(
    df,
    column_config={
        "Resume 📄": st.column_config.LinkColumn(
            display_text="📄 Open Resume"
        )
    },
    use_container_width=True
)
```

```
# ---------------- SEND MAIL ----------------
st.subheader("📧 Send Mail")

for r in st.session_state.results:

    col1, col2, col3 = st.columns([3,2,2])

    with col1:
        st.write(f"👤 {r['name']}")

    with col2:
        st.write(f"🎯 {r['match']} %")

    with col3:

        if st.button(
            f"Send Mail to {r['name']}",
            key=f"mail_{r['email']}"
        ):

            mail = requests.post(
                f"{API_URL}/send_mail/",
                json={
                    "email": r["email"],
                    "name": r["name"],
```

```
            "job_role": st.session_state.job_role
        }
    )

    if mail.status_code == 200:
        st.success(f"☑ Mail sent to {r['name']}")
    else:
        st.error("❌ Mail failed")
```

Parser.py :

```
import re

IT_ROLES = [
    "software engineer","softwaredeveloper","full stack developer",
    "backend developer","frontenddeveloper","web developer",
    "mobiledeveloper","android developer","ios developer",

    "python developer","javadeveloper","c++ developer",
    "dotnet developer","php developer","golang developer",

    "reactdeveloper","react native developer","angular developer",
    "vuedeveloper","nodejs developer","javascript developer",

    "data analyst","datascientist","data engineer",
    "mlengineer","ai engineer",
```

```
    "cloudengineer","cloud architect","aws engineer",
    "azure engineer","devops engineer",

    "cyber security analyst","security engineer",

    "qaengineer","testengineer","automation tester",

    "database administrator","sql developer",

    "ui designer","ux designer",

    "sapconsultant","salesforce developer",
    "power platform developer",

    "systemadministrator","network engineer",

    "productmanager","project manager",

    "blockchain developer","iot engineer"
]

# ==============================
# 📧 CONTACT INFO
# ==============================
```

```
def extract_contact_info(text):

    email_match = re.findall(
        r'[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}',
        text
    )
    email = email_match[0] if email_match else None

    phone_match = re.findall(
        r'(\+?\d[\d\s-]{8,14}\d)',
        text
    )

    phone = None
    if phone_match:
        phone = re.sub(r"\D", "", phone_match[0])

    return email, phone

# ==============================
# 💼 JOB ROLE EXTRACT
# ==============================
def extract_job_role(jd_text):

    lines = [l.strip() for l in jd_text.split("\n") if l.strip()]
```

```
    top_lines = lines[:5]  # only top area

    text = " ".join(top_lines).lower()

    # find roles ending with Developer/Engineer/Analyst
    match = re.search(
        r"\b([a-zA-Z]+\s(?:developer|engineer|analyst|designer))\b",
        text
    )

    if match:
        return match.group(1).title()

    return "Developer"
```

Matcher.py :

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

def match_percentage(jd_text, resume_text):

    if not jd_text or not resume_text:
        return 0.0
```

```
    vectorizer = TfidfVectorizer(stop_words="english", ngram_range=(1,2))

    try:
        tfidf = vectorizer.fit_transform([jd_text, resume_text])
        score = cosine_similarity(tfidf[0:1], tfidf[1:2])[0][0]
        return round(score * 100, 2)
    except:
        return 0.0
efextract_text_from_pdf(contents):
    text = ""
    doc = fitz.open(stream=contents, filetype="pdf")

    for page in doc:
        page_text = page.get_text()

        if page_text.strip():
            text += page_text
        else:
            pix = page.get_pixmap(matrix=fitz.Matrix(2,2))
            img = Image.open(io.BytesIO(pix.tobytes())).convert("L")
            text += pytesseract.image_to_string(img)

    return text

# ================= UPLOAD RESUME =================
```

```
@app.post("/upload_resume/")
async def upload_resume(name: str = Form(...), file: UploadFile = File(...)):

    contents = await file.read()
    unique_name = f"{uuid.uuid4()}_{file.filename}"

    with open(f"{UPLOAD_FOLDER}/{unique_name}", "wb") as f:
        f.write(contents)

    text = extract_text_from_pdf(contents) if file.filename.endswith(".pdf") else contents.decode()

    email, phone = extract_contact_info(text)

    db = SessionLocal()

    if not email:
        return {"msg":"Email not found"}

    existing = db.query(models.Resume).filter(
        (models.Resume.email == email) |
        (models.Resume.phone == phone)
    ).first()

    if existing:
        return {"msg":"Resume exists"}
```

```
    db.add(models.Resume(
        name=name,
        email=email,
        phone=phone,
        resume_text=text,
        file_path=unique_name
    ))

    db.commit()
    db.close()

    return {"msg":"Resume uploaded"}

# ================= MATCH JD =================
@app.post("/match_jd/")
async def match_jd(file: UploadFile = File(...)):

    contents = await file.read()
    jd_text = extract_text_from_pdf(contents) if file.filename.endswith(".pdf") else
contents.decode()

    job_role = extract_job_role(jd_text)
    jd_text_lower = jd_text.lower()
```

```
db = SessionLocal()
resumes = db.query(models.Resume).all()

results = []

for r in resumes:
    score = match_percentage(jd_text_lower, (r.resume_text or "").lower())

    results.append({
        "name": r.name,
        "email": r.email,
        "match": score,
        "resume_link": f"{PUBLIC_URL}/files/{r.file_path}",
        "phone": r.phone
    })

db.close()

results.sort(key=lambda x: x["match"], reverse=True)

for i,r in enumerate(results):
    r["rank"] = i+1

return {
    "detected_role": job_role,
```

```
    "results": results
}
```

9. BIBLIOGRAPHY AND REFERENCES :

- https://scikitlearn.org/stable/modules/generated/sklearn.metrics.pairwise.cosine_similarity.html
- https://www.sbert.net/
- https://pypi.org/project/sentence-transformers/
- https://fastapi.tiangolo.com/
- https://flask.palletsprojects.com/
- https://numpy.org/doc/
- https://pandas.pydata.org/docs/
- https://towardsdatascience.com/resume-matching-with-machine-learning-4f8b6f7c0b6e
- https://www.ibm.com/topics/natural-language-processing
- https://www.geeksforgeeks.org/nlp-natural-language-processing/